

TINKER: A Fully-Digital Threshold-Logic-Mapped BNN Inference SoC

Abdullah Sahruri and Martin Margala

School of Computing and Informatics, University of Louisiana at Lafayette, Lafayette, LA, USA
 {abdullah.sahruri1, martin.margala}@louisiana.edu

Abstract—This paper presents TINKER, a complete open-flow RISC-V inference SoC that integrates a sixteen-neuron binary neural network (BNN) tile with a PicoRV32 RV32I core, a Wishbone B4 fabric, and OpenRAM-based instruction/data memories in SkyWater sky130A. The tile implements XNOR/popcount/threshold neuron behavior through a fully digital threshold-logic mapping: the 64-bit threshold function is decomposed into chunked LUT sub-popcounts, an adder tree, and a comparator, all synthesized into ordinary standard cells. This avoids analog, current-mode, memristor, or flash primitives while preserving a portable RTL-to-GDS implementation path. TINKER runs a grouped 14×14 MNIST BNN through five software-scheduled tile evaluations and reaches 85.60 % accuracy on the full 10 000-image test set, with a 95 % Wilson confidence interval of [84.90 %, 86.27 %]. Post-P&R activity-aware evaluation reports 47.21 mW and 8.06 μ J/inference; the TLG tile accounts for 24.3 % of total SoC power, the largest active datapath bucket after the memory subsystem is implemented with OpenRAM macros. Physical verification is clean on KLayout DRC, Netgen LVS, KLayout XOR, and antenna. Timing closes at the headline TT corner; slow-corner macro timing is limited by OpenRAM’s single-corner Liberty model.

Index Terms—binary neural networks, threshold logic, system-on-chip, RISC-V, open-source EDA.

I. INTRODUCTION

Binary neural networks (BNNs) are attractive for small inference SoCs because multiplication collapses to bitwise XNOR and population count. The remaining question is not whether a BNN inner product can be synthesized, but whether a threshold-logic view of that inner product can be mapped into a complete, portable SoC implementation. Classical threshold logic fires when a weighted sum crosses a programmed threshold [1], yet prior silicon demonstrations commonly rely on fabrication-bound primitives such as current-mode summation [2], memristors [3], or embedded flash threshold devices [4]. Fig. 1 illustrates the conventional view: panel (a) is the abstract weighted-sum threshold neuron, and panel (b) is a representative capacitive realization that ties the threshold function to a specific analog primitive. Those approaches are useful architectural comparators, but they do not answer whether threshold-logic-mapped BNN compute can live inside an ordinary digital RTL-to-GDS flow. TINKER addresses this question with a fully digital threshold-logic-mapped BNN tile integrated into an open-flow RISC-V SoC. Each neuron computes XNOR/popcount/threshold behavior, but the 64-bit threshold function is implemented by a chunked-LUT decomposition: fixed-size LUT sub-popcounts

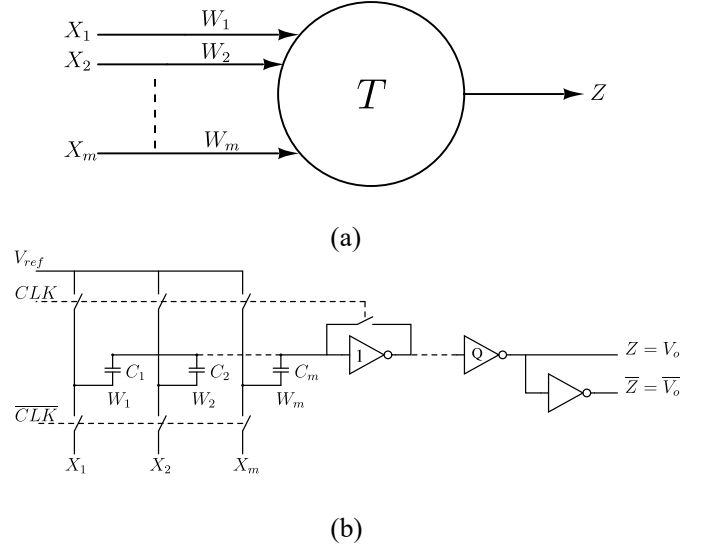


Fig. 1. Conventional threshold-logic neuron: (a) abstract weighted-sum-threshold model and (b) a representative capacitive realization [5] with charge redistribution onto weighted capacitors C_i followed by a clocked inverter chain. TINKER replaces this fabrication-bound primitive with a fully digital chunked-LUT mapping (Section III-B).

feed an adder tree and a comparator, all synthesized into `sky130_fd_sc_hd` standard cells. The tile is attached to a PicoRV32 [6] RV32I core over a Wishbone B4 fabric with OpenRAM IMEM/DMEM macros [7]. Firmware schedules a grouped 14×14 MNIST BNN through five sequential tile evaluations. This combination makes the work a hardware/software co-design and SoC-integration study, not an isolated accelerator macro.

The paper makes three contributions:

- a standard-cell chunked-LUT threshold-logic decomposition for BNN neurons, avoiding analog, current-mode, memristor, or flash primitives;
- a complete PicoRV32/Wishbone B4/OpenRAM inference SoC in the sky130A open RTL-to-GDS flow; and
- post-P&R activity-aware evaluation showing 85.60 % MNIST accuracy, 8.06 μ J/inference, and a TLG tile that accounts for 24.3 % of total SoC power, the largest active datapath bucket once register-file memories are replaced by OpenRAM macros.

II. BACKGROUND AND RELATED WORK

We organize prior work along three axes: (i) the device primitive used to realize the threshold function, (ii) the scope of the demonstration (gate, accelerator IP, or full SoC), and (iii) the openness of the implementation flow. TINKER occupies a specific cell in this space: a digital standard-cell primitive realized through chunked-LUT decomposition, integrated as a software-orchestrated SoC, in a fully open RTL-to-GDS flow.

A. Threshold-logic primitives

Most prior silicon TLG demonstrations bind the threshold function to a fabrication-specific primitive. Differential and current-mode designs realize the weighted sum electrically through summed branch currents [2], with compact gates but bias- and matching-dependent behavior that does not survive process migration. Capacitive realizations [5] encode weights as capacitor ratios with a precharge phase, with high fan-in variants adding offset compensation [8] or key-driven charge recycling for logic locking [9]. Memristor-based TLGs [3] use non-volatile resistance for both weight storage and analog summation, gaining device-level density at the cost of integration with mainstream digital flows. TULIP [4] is the closest threshold-logic ASIC comparator: it realizes a configurable BNN as a network of programmable threshold-logic standard cells whose threshold compare uses embedded flash, establishing the TLG-network view of BNN inference at accelerator scope but inheriting the process-portability limits of flash and reporting as a macro rather than a software-orchestrated SoC. The broader standard-cell threshold-logic synthesis literature [1] shares TINKER’s mapping style but targets gate- and netlist-level synthesis; the contribution here is one of scope, not new gate-level theory.

B. Digital BNN accelerators and open-flow SoCs

Digital BNN accelerators set the energy frontier for the XNOR-popcount-threshold datapath in commercial flows. YodaNN [10], XNE [11], and ChewBaccaNN [12] target 65–22 nm processes and report sub-microjoule per inference; FINN [13] demonstrates the same datapath in an FPGA dataflow framework. Closest to the mapping used here, TLG2LUT6 [14] folds binary weights and thresholds into LUT6 initialization strings, but is tied to a vendor fabric; the decomposition here targets standard cells and is carried to GDS inside a complete SoC. None of these targets a fully open-source RTL-to-GDS flow, and most report accelerator IP rather than a software-orchestrated SoC. The closest open-flow sky130 comparator is OpenSpike [15], which combines PicoRV32 and OpenRAM but targets a spiking network and does not address threshold-logic decomposition. The contribution here is therefore not raw efficiency at advanced nodes; it is an open, portable demonstration that a threshold-logic decomposition survives integration into a complete RISC-V SoC under software control. The chunked-LUT mapping introduced in Section III-B is the mechanism by which TINKER meets that constraint within an ordinary digital RTL-to-GDS flow.

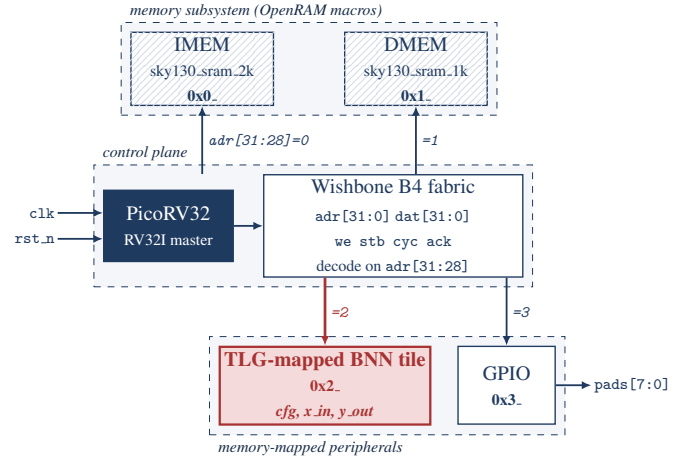


Fig. 2. SoC architecture block diagram: PicoRV32 master, Wishbone B4, OpenRAM IMEM/DMEM, TLG-mapped tile, and GPIO.

III. SOC ARCHITECTURE

A. System overview

TINKER is a single-clock RV32I system that hosts and orchestrates the TLG-mapped tile under software control. A PicoRV32 core [6] acts as the lone Wishbone B4 classic master and drives a single-cycle bus with 32-bit address and 32-bit data; address decode on $\text{adr}[31:28]$ dispatches to one of four word-aligned slaves: IMEM at $0x0000_0000$, DMEM at $0x1000_0000$, the tile peripheral at $0x2000_0000$, and a sim-observability GPIO at $0x3000_0000$. Reset is asynchronously asserted and synchronously deasserted through a two-stage flop chain; firmware boots bare-metal from $\text{PROGADDR_RESET} = 0$. A single-master, four-slave decode is sufficient for the polled inference handshake. The system clock is nominally 100 MHz, matching the standalone tile constraint. Fig. 2 shows the block diagram.

B. Threshold-logic decomposition and tile microarchitecture

a) *Neuron function*: Each tile neuron computes the binary inner product $y_i = \mathbf{1}[\text{popcount}(x \oplus w_i) \geq \theta_i]$, with $x \in \{-1, +1\}^{64}$ and per-neuron 64-bit weight w_i and 7-bit threshold θ_i . This realizes threshold-function behavior without instantiating a literal high-fanin physical threshold gate. Sixteen neurons share the registered input and produce a 16-bit output vector per evaluation, with two-cycle latency. Fig. 3 shows the tile datapath and configuration path.

b) *Chunked-LUT decomposition*: The decomposition recasts the 64-input threshold function as a small adder-tree of population counts over fixed-size chunks. The 64-bit XNOR output is partitioned into eleven chunks: ten 6-bit chunks plus one 4-bit residue. Each 6-bit chunk’s population count is a small Boolean function ($6 \rightarrow 3$) that Yosys synthesises into a network of `sky130_fd_sc_hd` multiplexers and AOI cells; the 4-bit residue ($4 \rightarrow 3$) is similarly decomposed. The eleven 3-bit subcounts feed a conventional width-7 adder tree, and the 7-bit sum is compared against θ_i by a standard magnitude comparator. Fig. 4 visualises the decomposition.

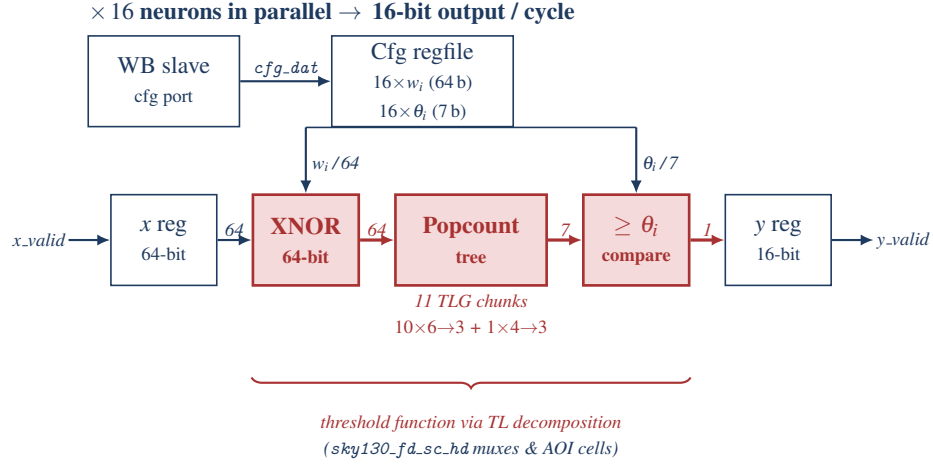


Fig. 3. TLG-mapped BNN tile microarchitecture. Sixteen XNOR-popcount-threshold neurons share the registered input, use eleven chunk-LUT sub-popcounts per neuron, and produce a 16-bit output vector.

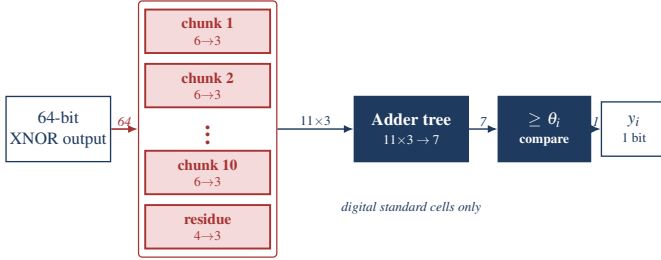


Fig. 4. Chunked-LUT decomposition of the 64-bit threshold function. The XNOR output partitions into ten 6-bit chunks and one 4-bit residue; each sub-popcount is synthesized into `sky130_fd_sc_hd` logic. The subcounts feed an adder tree and threshold comparator.

The chunked approach bounds fanin and avoids a fabrication-specific summation primitive. The 6-bit chunk size matches `sky130_fd_sc_hd`'s mux/AOI cost: large enough that the eleven-way partitioning overhead is acceptable, small enough that each chunk LUT fits comfortably as a combinational cell network.

c) Programmability: The configuration regfile holds 16 weight words (64-bit) and 16 thresholds (7-bit), addressed through a 6-bit `cfg_addr`; weights are loaded once per network and the `cfg` port carries traffic only at boot. An `x_valid` strobe latches the input register and starts the two-cycle pipeline; `y_valid` pulses on the cycle the output leaves.

C. Tile wrapper and inference handshake

The Wishbone-to-tile bridge `wb_tile_wrapper` occupies a 512-byte slave window and exposes the tile's `cfg` port and `x_in/y_out` handshake as memory-mapped registers. Weights and inputs are committed by two-phase 32-bit writes (LO then HI), with the HI write assembling the 64-bit value and pulsing the corresponding strobe; thresholds are atomic single-word writes. Firmware polls `STATUS.BUSY` until clear, reads `YOUT`, and writes `CTRL.CLR_DONE` to arm the next inference. Writes to weight, threshold, or input registers while

`BUSY = 1` are silently dropped: the bus access acks normally but no shadow is updated, causing a misbehaving program to hang visibly rather than corrupt an in-flight evaluation. This is a deliberate testability choice: a silently corrupted inference is a far worse failure mode than an observably hung firmware loop. The wrapper carries roughly 1.3 K flops, ~ 700 of them purely for read-back of every writable register.

The memory subsystem instantiates pre-hardened OpenRAM [7] 1RW1R macros: 2 KB for IMEM and 1 KB for DMEM. A small Wishbone FSM wraps each macro and inserts one wait state on reads. The motivation for OpenRAM macros over a register-file baseline is power; the system-level consequence is detailed in Section VI-B.

IV. WORKLOAD MAPPING AND DEPLOYMENT

The deployed network is a grouped binary MLP over a 14×14 input. After a 2×2 average pool and a per-quadrant median binarisation, four parallel branches each map a 49-bit quadrant to 16 hidden bits in one tile evaluation; the four 16-bit outputs concatenate into a 64-bit hidden vector that the same tile maps to ten output logits in a fifth evaluation. Grouping (rather than full $196 \rightarrow 64$ connectivity) is a hardware-shape choice: each branch fits one tile-evaluation fanin, so the entire network maps onto the SoC without changing tile or wrapper RTL. The accuracy cost of grouping was bounded before deployment with a float-precision proxy: trained on the same data with the same preprocessing, a grouped $4 \times (49 \rightarrow 16)$ model trails an unconstrained $196 \rightarrow 64$ model by 0.87 pp. That gap is the same order as the deployed network's own experimental spread—0.74 pp across the three training seeds, and a 0.69 pp half-width on the 95 % Wilson confidence interval—and an order below the accuracy given up to binarisation itself. Per-image inference therefore costs four hidden-layer batches plus one output-layer batch, which the PicoRV32 firmware schedules sequentially against the `cfg-write/poll/read` protocol of Section III-C, totalling 17 068 cycles or 170.7 μ s at 100 MHz. The bare-metal firmware compiles to roughly

1.5KB of RV32I and shares the IMEM with the binary weights; per-image input bits are preloaded into DMEM by the testbench at simulation time zero. A vectorised Python integer model serves as the deployment golden, folding the trained batch-norm parameters into per-neuron thresholds via $t_i = \lceil (49 + \text{sgn}(\gamma_i) \mu_i - \beta_i \sigma_i / |\gamma_i|) / 2 \rceil$; SoC RTL simulation matches this golden on all sixteen verification images. Weights are trained off-chip with binary-weight gradient descent [16] and a hard-tanh straight-through estimator [17] on a OneCycleLR schedule [18].

V. IMPLEMENTATION

A. Open-flow toolchain

The SoC is implemented through the LibreLane 3.0.3 RTL-to-GDSII flow [19] on the SkyWater sky130A open-source PDK. Yosys synthesises the design against the `sky130_fd_sc_hd` standard-cell library; OpenROAD [20] drives floorplanning, placement, clock-tree synthesis, routing, and antenna repair; Magic and KLayout perform GDS streamout and DRC; Netgen performs LVS. The OpenRAM IMEM and DMEM macros occupy $683 \times 417 \mu\text{m}$ and $480 \times 398 \mu\text{m}$, respectively, and are manually placed in a $1500 \times 1500 \mu\text{m}$ absolute floorplan with PDN hookups to the sky130 dual rail. Macro integration required iteration on placement constraints, an OpenSTA blackbox stub for the behavioural macro views, and removal of the wrapper-level `keep_hierarchy` attributes to satisfy LibreLane’s flat-macro convention. The full RTL-to-GDSII pipeline uses one flow configuration for multi-corner STA, physical verification, and activity-aware power decomposition.

B. Sign-off methodology

Fig. 5 shows the post-P&R floorplan, with the IMEM and DMEM macros along the upper edge of the die and the std-cell logic below. Multi-corner static timing was run across nine corners covering the three sky130 process points (TT, FF, SS), three temperatures (-40°C , 25°C , 100°C), and three supply voltages (1.60 V, 1.80 V, 1.95 V). At the headline `nom_tt_025C_1v80` corner the design closes with $+0.51\text{ns}$ setup slack and $+0.37\text{ns}$ hold slack; hold timing is clean across all nine corners. Slow-corner setup violations at `nom_ss_100C_1v60` are an attribute of the OpenRAM macros’ single TT-corner Liberty, a documented LibreLane 3.x limitation discussed in Section VII, not a design defect. Physical verification is clean on KLayout DRC, Netgen LVS, KLayout XOR, and antenna: respectively 0 errors, 0 errors, 0 differences, and 0 violating nets/pins. The Magic DRC flags inside hardened SRAM macro footprints are excluded from this claim because they are a known sky130 macro-view issue.

VI. EVALUATION

A. Functional verification

The deployed network is exercised through four verification gates, summarised in Table I. The vectorised Python integer model is treated as the deployment golden, since its arithmetic is exact in the same fixed-width integer domain as the on-chip

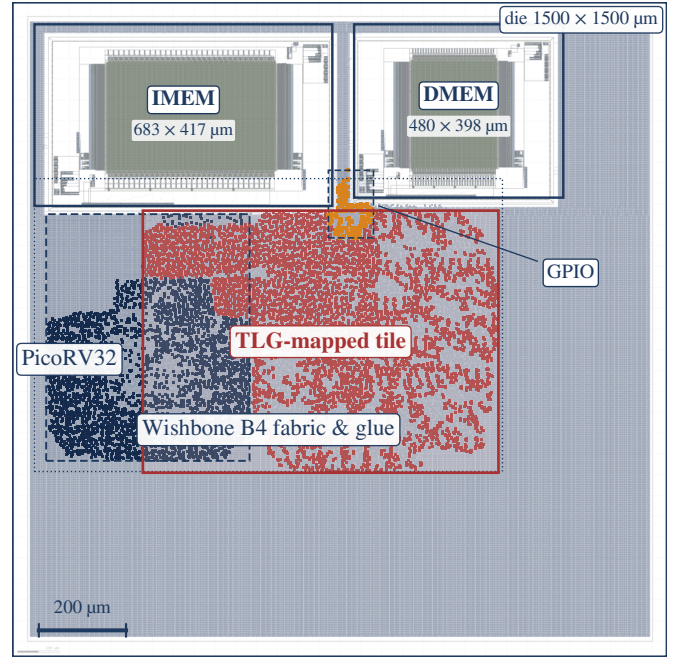


Fig. 5. Post-P&R floorplan of the open-flow BNN inference SoC on sky130A ($1.5 \times 1.5 \text{ mm}^2$ die). The IMEM and DMEM OpenRAM macros sit in the upper region; module footprints for the TLG-mapped tile, PicoRV32 core, Wishbone B4 fabric, and GPIO are extracted from the placed instance bounding boxes.

TABLE I
FUNCTIONAL VERIFICATION SUMMARY.

Gate	Reference	Result
PyTorch $\leftrightarrow$ integer golden	integer model	15/16 (1 boundary)
SoC sim $\leftrightarrow$ integer golden	RTL simulation	16/16 PASS
Full MNIST 10 K eval	vectorized XNOR-popcount	85.60 % [84.90, 86.27]
Sign-off DRC / LVS / antenna	KLayout / Netgen	0 / 0 / 0

XNOR-popcount; the trained PyTorch model is a training-time proxy whose floating-point activations may differ near batch-norm thresholds. The 16-image verification set is the first 16 images of the MNIST test split, used as a fast bit-exact check; the full 10000-image test set is covered by the integer-golden gate. PyTorch and the integer golden agree on 15 of these 16 images. The single mismatch lies on a folded-batch-norm activation boundary near zero and is the expected behaviour of the integer quantisation, not a bug. The post-P&R SoC RTL simulation then matches the integer golden bit-exact on all sixteen images. On the full 10000-image MNIST test set [21], the same network reaches 85.60 % accuracy (8560 correct), with a 95 % Wilson confidence interval of [84.90 %, 86.27 %]; per-class accuracy ranges from 95.2 % on digit 1 to 75.1 % on digit 5.

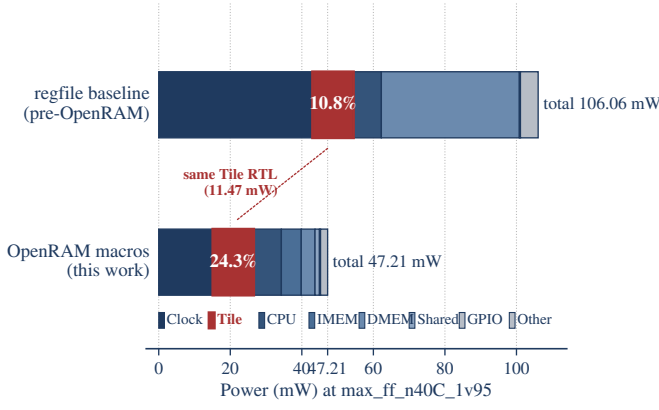


Fig. 6. Hierarchical activity-aware power decomposition of the open-flow SoC at `max_ff_n40C_1v95` (total 47.21 mW). The TLG-mapped tile is the largest active datapath bucket once the OpenRAM macros remove register-file dominance from the memory subsystem.

B. Activity-aware power decomposition

Activity-aware power is reported at the headline `max_ff_n40C_1v95` corner. The full sixteen-image inference testbench is replayed against the post-P&R netlist, the chip-pin VCD is annotated through OpenSTA’s `read_vcd`, supplemented with forty-seven named-net activities for the major interior wires, and `report_power` is decomposed into per-instance buckets by a bucketization pass. Total post-P&R power is 47.21 mW. Fig. 6 shows the hierarchical decomposition. The clock tree is the largest single contributor at 15.10 mW (32.0%). The TLG-mapped tile is the largest active datapath bucket at 11.47 mW (24.3%); the PicoRV32 core draws 7.71 mW (16.3%); the IMEM and DMEM macros draw 5.56 mW (11.8%) and 3.85 mW (8.2%) respectively; shared logic and the GPIO slave account for 1.24 mW (2.6%) and 0.30 mW (0.6%).

The actionable insight is the change in the tile’s share of the budget. With the register-file IMEM/DMEM of the Phase 3 baseline, the same tile RTL (byte-identical post-flatten netlist) contributed 11.47 mW of an SoC total of 106.06 mW, or 10.81%. Replacing the register files with OpenRAM macros drops the system total to 47.21 mW while leaving the tile bucket unchanged, lifting the tile’s share to 24.29% and dropping the memory subsystem’s combined share from 36.4% to 19.9%. This share is not, by itself, an efficiency claim; it is architectural evidence that compute remains load-bearing at SoC scale after the memory subsystem is converted from register files to OpenRAM macros.

C. Energy and throughput

At the same `max_ff` corner, the SoC executes 17068 cycles per image, i.e. 170.7 μ s and **8.06 μ J per inference** at 100 MHz. This corresponds to **124 113 inferences/J** and a sustained throughput of 5 860 inf/s under continuous polling. The companion `nom_tt_025C_1v80` corner is slightly more efficient at an estimated 7.09 μ J/image and 141 044 inf/J. Compared with the Phase 3 7×7 deployment on the same SoC,

TABLE II
SAME-FLOW STANDALONE TILE ABLATION. BOTH LOADABLE VARIANTS USE THE SAME LIBRELANE TILE FLOW, SDC, AND 100 MHz TARGET; ONLY THE NEURON RTL DIFFERS. POWER IS DEFAULT-ACTIVITY OPENSTA AT THE NOMINAL CORNER.

Metric	Chunked-LUT TLG	Adder-tree
Cells	22 125	21 464
Area (μ m ²)	320 367	305 300
Slow Fmax (MHz)	73.81	77.51
Power (μ W)	64 566	167 858

TABLE III
BNN/THRESHOLD-LOGIC ACCELERATOR POSITIONING. DESIGNS DIFFER IN NODE, NETWORK, VOLTAGE, AND REPORTING SCOPE; THE TABLE IS NOT A DIRECT EFFICIENCY RANKING. [†] INHERITED TRAINING-METHOD ACCURACY; [‡] CORE DYNAMIC ENERGY ONLY.

Ref.	Tech.	Network	Acc.	E/inf.	Area
[11]	22 nm	ResNet-18 IN	51.2 %	1 450 μ J	2.32 mm ²
[10]	UMC 65 nm	AlexNet IN	$\sim$ 56 % [†]	352 μ J	1.9 mm ²
[4]	40 nm	BinaryNet C10	89.85 % [†]	184 μ J	1.8 mm ²
[12]	GF 22 nm FDX	VGG-like C10	86.8 %	1.3 μ J	0.7 mm ²
[15]	sky130	1024-1024-10 SNN	99.12 %	$\sim$ 2.5 μ J [‡]	33.3 mm ²
This work	sky130A	14 $\times$ 14 BNN	85.60 %	8.06 μJ	2.25 mm ²

the grouped 14×14 network gains +14.48 pp of accuracy at +1.0% energy overhead per image: the post-P&R netlist, sign-off artefacts, and clock are byte-identical, and only the firmware DMEM layout (8 vs 2 words/image) and the binary weight image change between the two deployments. The accuracy-per-microjoule figure of merit therefore rises from 8.91 pp/ μ J to 10.62 pp/ μ J, a 19.2% improvement at unchanged silicon.

D. Tile ablation

A same-flow standalone tile ablation compares the proposed chunked-LUT TLG tile with a conventional synthesized XNOR-popcount-comparator tile generated by the same scripts and run through the same LibreLane tile flow. Table II shows the loadable variants, which include the programmable weight and threshold register file needed by the SoC. The TLG mapping is slightly larger and slower in this standalone flow, but reports lower default-activity power. Because this ablation is tile-level and uses OpenSTA default activity, the SoC-level claims in this paper rely on the measured integrated TLG design rather than on a projected replacement tile.

E. Comparison with prior BNN ASICs

Table III positions the proposed SoC against representative BNN and threshold-logic ASICs. The table spans different nodes, networks, voltages, memory systems, and reporting scopes, so it is intended for positioning rather than direct superiority claims. XNE [11], YodaNN [10], and ChewBaccaNN [12] are advanced-node commercial-flow BNN accelerators; TULIP [4] is the closest threshold-logic ASIC comparator but uses programmable flash devices; OpenSpike [15] is the closest open-flow sky130 SoC comparator but targets an SNN.

The distinguishing point for this work is the combination of sky130A open flow, OpenRAM memories, SoC-level reporting, and a fully digital standard-cell TLG mapping.

VII. DISCUSSION

The most consequential finding is the change in where SoC power is spent. The OpenRAM substitution reported in Section VI-B leaves the tile bucket byte-identical yet promotes it from 10.81 % of the budget to 24.29 % by removing register-file dominance from the memory subsystem. The system-integration consequence is that BNN compute is no longer hidden by storage overhead at SoC scale, and tile-level optimisation becomes a first-order target rather than a rounding error against IMEM/DMEM dominance.

The reported per-inference energy of 8.06 μ J should be read in its technology and scope context. Advanced-node commercial-flow BNN accelerators operate at smaller geometries and often report IP blocks rather than full SoCs; TINKER instead emphasizes portability, open-flow implementation, and system integration. The same-flow tile ablation in Table II helps isolate the chunked-LUT decomposition at tile level, but a full SoC re-integration of the conventional comparator tile is still useful future work.

The sign-off result is also deliberately bounded. Physical verification is clean on KLayout DRC, Netgen LVS, KLayout XOR, and antenna. Timing closes at the headline `nom_tt_025C_1v80` corner, while slow-corner macro timing is limited by OpenRAM's single-corner Liberty model. The present design does not demonstrate a multi-frequency power sweep, a larger network beyond grouped 14×14 , or a tape-out.

VIII. CONCLUSION

We have presented TINKER, a fully-digital threshold-logic-mapped BNN inference SoC in sky130A. The tile maps the XNOR/popcount/threshold neuron into standard cells with a chunked-LUT TLG decomposition and integrates it with a PicoRV32 RV32I core, Wishbone B4 fabric, and OpenRAM IMEM/DMEM macros. The grouped MNIST BNN reaches 85.60 % accuracy at 8.06 μ J/inference, and activity-aware post-P&R analysis shows the tile at 24.3 % of total SoC power. Physical verification is clean on KLayout DRC, Netgen LVS, KLayout XOR, and antenna; TT timing closes, while slow-corner macro timing is limited by OpenRAM's single-corner Liberty model.

REFERENCES

- [1] V. Beiu, J. M. Quintana, and M. J. Avedillo, "VLSI implementations of threshold logic—a comprehensive survey," *IEEE Transactions on Neural Networks*, vol. 14, no. 5, pp. 1217–1243, Sep. 2003.
- [2] V. Beiu, J. M. Quintana, M. J. Avedillo, and R. Andonie, "Differential implementations of threshold logic gates," in *Proceedings of the IEEE International Symposium on Signals, Circuits and Systems*, vol. 2, Iasi, Romania, Jul. 2003, pp. 489–492.
- [3] G. Papandroulidakis, A. Serb, A. Khiat, G. V. Merrett, and T. Prodromakis, "Practical implementation of memristor-based threshold logic gates," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 66, no. 8, pp. 3041–3051, Aug. 2019.
- [4] A. Wagle, S. P. Khatri, and S. B. K. Vrudhula, "A configurable BNN ASIC using a network of programmable threshold logic standard cells," in *2020 IEEE 38th International Conference on Computer Design (ICCD)*, 2020, pp. 433–440.
- [5] H. Özdemir, A. Kepkep, B. Pamir, Y. Leblebici, and U. Çilingiroglu, "A capacitive threshold-logic gate," *IEEE Journal of Solid-State Circuits*, vol. 31, no. 8, pp. 1141–1150, Aug. 1996.
- [6] C. X. Wolf, "PicoRV32: A size-optimized RISC-V CPU," <https://github.com/YosysHQ/picorv32>, 2015, accessed: 2026-05-04.
- [7] M. R. Guthaus, J. E. Stine, S. Ataei, B. Chen, B. Wu, and M. Sarwar, "OpenRAM: An open-source memory compiler," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2016, pp. 1–6.
- [8] A. Sahruri, M. Margala, and U. Çilingiroglu, "HiCTL: High fan-in differential capacitive-threshold-logic gate implementation with an offset-compensated comparator," in *2024 25th International Symposium on Quality Electronic Design (ISQED)*, 2024, pp. 1–7.
- [9] A. Sahruri and M. Margala, "TLGLock: A new approach in logic locking using key-driven charge recycling in threshold logic gates," in *2025 IFIP/IEEE 33rd International Conference on Very Large Scale Integration (VLSI-SoC)*, 2025, pp. 1–5.
- [10] R. Andri, L. Cavigelli, D. Rossi, and L. Benini, "YodaNN: An architecture for ultralow power binary-weight CNN acceleration," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 1, pp. 48–60, Jan. 2018.
- [11] F. Conti, P. D. Schiavone, and L. Benini, "XNOR neural engine: A hardware accelerator IP for 21.6-fJ/op binary neural network inference," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 11, pp. 2940–2951, Nov. 2018.
- [12] R. Andri, G. Karunaratne, L. Cavigelli, and L. Benini, "ChewBaccaNN: A flexible 223 TOPS/W BNN accelerator," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2021, pp. 1–5.
- [13] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, "FINN: A framework for fast, scalable binarized neural network inference," in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2017, pp. 65–74.
- [14] A. Sahruri and M. Margala, "TLG2LUT6: Threshold logic mapping for sub-nanosecond binary neural network inference," in *Proceedings of the Great Lakes Symposium on VLSI 2026 (GLSVLSI)*, 2026, pp. 103–108.
- [15] F. Modaresi, M. R. Guthaus, and J. K. Eshraghian, "OpenSpike: An OpenRAM SNN accelerator," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2023, pp. 1–5.
- [16] M. Courbariaux, Y. Bengio, and J.-P. David, "BinaryConnect: Training deep neural networks with binary weights during propagations," in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 3123–3131.
- [17] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks," in *Advances in Neural Information Processing Systems*, vol. 29, 2016, pp. 4107–4115.
- [18] L. N. Smith and N. Topin, "Super-convergence: Very fast training of neural networks using large learning rates," 2017.
- [19] The LibreLane Contributors, "LibreLane: ASIC implementation flow infrastructure," <https://github.com/librelane/librelane>, 2025, version 2.4.13; accessed: 2026-05-04.
- [20] I. Hubara, V. A. Chhabria, M. Fogaça, S. Hashemi, A. Hosny, A. B. Kahng, M. Kim, J. Lee, U. Mallappa, M. Neseem, G. Pradipta, S. Reda, M. Saligane, S. S. Sapatnekar, C. Sechen, M. Shalan, W. Swartz, L. Wang, Z. Wang, M. Woo, and B. Xu, "Toward an open-source digital flow: First learnings from the OpenROAD project," in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 76:1–76:4.
- [21] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.